Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №_

«Название лабораторной работы»

Выполнил:
студент группы …
Беспалов Р. Т.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 3
2. Руководство пользователя 5
3. Руководство программиста 5
3.1. Описание структуры программы 7
3.2. Описание структур данных 7
3.3. Описание алгоритмов 8
4. Результаты экспериментов и тестирования 8
Заключение 10
Литература 12
Приложение 14

Введение

Цель данной работы — разработка структуры данных для хранения множеств с использованием битовых полей, а также освоение фрэймворка для разработки автоматических тестов Google Test.

В рамках работы решаются следующие задачи:

1. Реализация класса битового поля TBitField согласно заданному интерфейсу.
2. Реализация класса множества TSet согласно заданному интерфейсу.
3. Написание и обеспечение работоспособности тестов, покрывающих основной функционал и краевые случаи.

1. Постановка задачи

Необходимо реализовать два класса:

TBitField — битовое поле, представляющее собой набор битов с номерами от 0 до BitLen-1. Внутренне поле хранится в массиве элементов типа TELEM (unsigned int), где биты упакованы по несколько штук в один элемент. Требуется реализовать:

1. конструктор, конструктор копирования, деструктор;
2. методы доступа к битам: GetLength, SetBit, ClrBit, GetBit;
3. битовые операции: ==, !=, =, |, &, ~;
4. операторы ввода/вывода >> и <<.

TSet — множество, хранящееся в виде характеристического вектора (битового поля). Требуется реализовать:

1. конструкторы (обычный, копирования, преобразования из TBitField);
2. преобразование к TBitField;
3. методы доступа: GetMaxPower, InsElem, DelElem, IsMember;
4. теоретико-множественные операции: ==, !=, =, + (с элементом), - (с элементом), + (объединение), * (пересечение), ~ (дополнение);
5. операторы ввода/вывода.

2. Руководство пользователя

Программа представляет собой библиотеку классов, предназначенную для использования в других программах. Проверка корректности работы реализованных классов осуществляется с помощью набора автоматических тестов, написанных с использованием фреймворка Google Test. Консольное приложение в рамках данной работы не разрабатывается; взаимодействие с классами происходит исключительно через тестовые сценарии.

2.1. Запуск тестов

Для сборки и запуска тестов необходимо:

- Открыть решение (solution) в Microsoft Visual Studio.
- Убедиться, что проект с тестами подключён к решению и содержит файл test.cpp с тестами.
- Собрать решение (Build → Build Solution).
- Запустить тесты через Test Explorer (Test → Run All Tests) или запустить исполняемый файл тестового проекта из командной строки.

2.2. Использование тестов Google Test

Тесты оформлены в виде отдельных макросов TEST(TestSuiteName, TestName), сгруппированных в два набора:

- TBitField — тесты для класса битового поля;
- TSet — тесты для класса множества.

Внутри каждого теста выполняются:

- создание объектов классов с различными параметрами;
- вызов проверяемых методов;
- проверка результатов с помощью макросов ASSERT_EQ, ASSERT_TRUE, ASSERT_FALSE, ASSERT_THROW, ASSERT_NO_THROW.

2.3. Проверяемые сценарии

Тесты покрывают следующие категории проверок:

- Корректность создания объектов — конструкторы с допустимыми и недопустимыми параметрами (отрицательная длина/мощность, нулевая длина, конструктор копирования, конструктор преобразования из TBitField).
- Работа с отдельными битами и элементами — установка, очистка и проверка битов; включение, удаление и проверка принадлежности элементов; границы элементов массива TELEM (биты 0, 31, 32, 39, 63, 64).
- Обработка ошибочных ситуаций — генерация исключений std::invalid_argument и std::out_of_range при некорректных индексах, отрицательных размерах и операциях над множествами с разными универсальными множествами.
- Битовые и теоретико-множественные операции — сравнение (==, !=), присваивание (=), объединение (|, +), пересечение (&, *), отрицание/дополнение (~), объединение и разность с элементом.
- Проверка независимости копий — изменение копии не влияет на оригинал (конструктор копирования, оператор присваивания).
- Проверка самоприсваивания — безопасность операции x = x.

- Операции ввода/вывода — чтение строки '0'/'1' для TBitField, чтение чисел для TSet, вывод в поток; обработка некорректного ввода.

2.4. Интерпретация результатов

После запуска тестов в Test Explorer отображается список всех тестов с результатами:

- зелёная галочка — тест пройден;
- красный крестик — тест не пройден (с указанием строки и причины).

Все 47 тестов должны быть пройдены успешно. При наличии падений необходимо проанализировать сообщение об ошибке, определить нарушенное утверждение и исправить реализацию соответствующего метода.

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из следующих модулей:

Модуль	Назначение
tbitfield.h	Объявление класса TBitField (интерфейс, не изменяется)
tset.h	Объявление класса TSet (интерфейс, не изменяется)
tbitfieldR.h	Реализация методов класса TBitField
tsetR.h	Реализация методов класса TSet
test.cpp	Тесты Google Test

3.2. Описание структур данных

3.2.1. Класс TBitField

Битовое поле хранится в массиве элементов типа TELEM (unsigned int, обычно 32 бита). Бит с номером n находится в элементе с индексом n / BITS_PER_CHUNK на позиции n % BITS_PER_CHUNK (счёт справа налево, от младших к старшим). Количество элементов вычисляется как MemLen = BitLen / BITS_PER_CHUNK + (BitLen % BITS_PER_CHUNK != 0).

Закрытые поля:

- int BitLen — длина битового поля (максимальное количество битов);
- TELEM *pMem — указатель на массив элементов для хранения битов;
- int MemLen — количество элементов в массиве pMem.

Закрытые методы:

- int GetMemIndex(const int n) const — возвращает индекс элемента массива для бита n;
- TELEM GetMemMask(const int n) const — возвращает битовую маску для бита n.

Открытые методы: конструкторы, деструктор, GetLength, SetBit, ClrBit, GetBit, операторы сравнения, присваивания, битовых операций и ввода/вывода.

3.2.2. Класс TSet

Множество представляется характеристическим вектором — битовым полем длины MaxPower, где бит i равен 1, если элемент i принадлежит множеству, и 0 иначе.

Закрытые поля:

- int MaxPower — максимальная мощность множества (размер универсального множества);
- TBitField BitField — битовое поле для хранения характеристического вектора.

Открытые методы: конструкторы, преобразование к TBitField, GetMaxPower, InsElem, DelElem, IsMember, теоретико-множественные операции, ввод/вывод.

3.3. Описание алгоритмов

3.3.1. Класс TBitField

Метод	Описание	Сложность
TBitField(int len)	Вычисляет MemLen, выделяет память и обнуляет её. Проверяет len >= 0.	O(MemLen)
TBitField(const TBitField&)	Копирует BitLen, MemLen и массив pMem.	O(MemLen)
~TBitField()	Освобождает память delete[] pMem.	O(1)
GetMemIndex(n)	Возвращает n / BITS_PER_CHUNK.	O(1)
GetMemMask(n)	Возвращает 1 << (n % BITS_PER_CHUNK).	O(1)
GetLength()	Возвращает BitLen.	O(1)
SetBit(n)	Проверяет индекс, выполняет pMem[GetMemIndex(n)] |= GetMemMask(n).	O(1)
ClrBit(n)	Проверяет индекс, выполняет pMem[GetMemIndex(n)] &= ~GetMemMask(n).	O(1)
GetBit(n)	Проверяет индекс, возвращает (pMem[GetMemIndex(n)] & GetMemMask(n)) != 0.	O(1)
operator==	Побитовое сравнение с учётом разной длины (недостающие биты считаются нулевыми).	O(max(BitLen))
operator!=	Отрицание operator==.	O(max(BitLen))
operator=	Проверка самоприсваивания, выделение новой памяти, копирование, освобождение старой.	O(MemLen)
operator|	Создаёт результат длины max(BitLen), побитово выполняет ИЛИ.	O(max(BitLen))
operator&	Создаёт результат длины max(BitLen), побитово выполняет И.	O(max(BitLen))
operator~	Создаёт результат той же длины, инвертирует все биты.	O(BitLen)
operator>>	Читает BitLen символов '0'/'1', при неверном символе устанавливает failbit.	O(BitLen)
operator<<	Выводит BitLen символов '0'/'1'.	O(BitLen)

3.3.2. Класс TSet

Метод	Описание	Сложность
TSet(int mp)	Инициализирует MaxPower = mp, BitField(mp). Проверяет mp >= 0.	O(MemLen)
TSet(const TSet&)	Копирует MaxPower и BitField.	O(MemLen)
TSet(const TBitField&)	Инициализирует MaxPower = bf.GetLength(), копирует BitField.	O(MemLen)
operator TBitField()	Возвращает копию BitField.	O(MemLen)
GetMaxPower()	Возвращает MaxPower.	O(1)
InsElem(Elem)	Проверяет диапазон, вызывает BitField.SetBit(Elem).	O(1)
DelElem(Elem)	Проверяет диапазон, вызывает BitField.ClrBit(Elem).	O(1)
IsMember(Elem)	Проверяет диапазон, возвращает BitField.GetBit(Elem).	O(1)
operator==	Сравнивает MaxPower и BitField.	O(MemLen)
operator!=	Отрицание operator==.	O(MemLen)
operator=	Копирует MaxPower и BitField.	O(MemLen)
operator+(int)	Создаёт копию, включает элемент.	O(MemLen)
operator-(int)	Создаёт копию, удаляет элемент.	O(MemLen)
operator+(TSet)	Проверяет равенство MaxPower, создаёт результат, выполняет BitField | s.BitField.	O(MemLen)
operator*(TSet)	Проверяет равенство MaxPower, создаёт результат, выполняет BitField & s.BitField.	O(MemLen)
operator~	Создаёт результат, выполняет ~BitField.	O(MemLen)
operator>>	Читает целые числа, устанавливает соответствующие биты.	O(k), k — число элементов
operator<<	Выводит индексы установленных битов через пробел.	O(MaxPower)

4. Результаты экспериментов и тестирования

Тестирование проводилось с использованием фреймворка Google Test. Написано 47 тестов, покрывающих основной функционал и краевые случаи.

4.1. Тесты класса TBitField

№	Тест	Проверяемый сценарий
1	ConstructorSetsLengthAndClearsBits	Конструктор устанавливает длину и обнуляет все биты
2	ConstructorRejectsNegativeLength	Отрицательная длина вызывает std::invalid_argument
3	ZeroLengthIsAllowed	Нулевая длина допустима
4	SetBitSetsOnlyRequestedBit	Установка битов на границах элементов (0, 31, 32, 39)
5	SetBitThrowsInvalidIndex	Некорректный индекс вызывает std::out_of_range
6	ClrBitClearsRequestedBit	Очистка битов на границах элементов
7	ClrBitThrowsInvalidIndex	Некорректный индекс вызывает std::out_of_range
8	GetBitThrowsForInvalidIndex	Некорректный индекс вызывает std::out_of_range
9	CopyConstructorCreatesIndependentCopy	Копия независима от оригинала
10	AssignmentCreatesIndependentCopy	Присваивание создаёт независимую копию, длина изменяется
11	SelfAssignmentIsSafe	Самоприсваивание безопасно
12	EqualityConsidersLengthAndBits	Сравнение учитывает длину и биты (разная длина, но одинаковые используемые биты — равны)
13	OperatorOrUnionsBits	Операция ИЛИ
14	OperatorAndIntersectsBits	Операция И
15	BitwiseOperatorsAcceptDifferentLengths	Операции И/ИЛИ работают с разной длиной
16	OperatorNotInvertsOnlyUsedBits	Отрицание инвертирует только используемые биты
17	OperatorNotDoesNotChangeOriginal	Отрицание не меняет оригинал
18	DoubleNotReturnsOriginalBits	Двойное отрицание возвращает исходное поле
19	InputStreamReadsBinaryDigits	Ввод строки '0'/'1'
20	InputStreamRejectsInvalidCharacter	Неверный символ устанавливает failbit
21	OutputStreamPrintsBits	Вывод строки '0'/'1'

4.2. Тесты класса TSet

№	Тест	Проверяемый сценарий
22	ConstructorSetsMaxPowerAndEmptySet	Конструктор устанавливает мощность и пустое множество
23	ConstructorRejectsNegativeMaxPower	Отрицательная мощность вызывает std::invalid_argument
24	ConstructorFromBitFieldCopiesLengthAndBits	Конструктор из TBitField копирует длину и биты
25	ConversionToBitFieldReturnsSameBits	Преобразование к TBitField возвращает те же биты
26	InsElemAddsElement	Включение элемента (границы 0 и 9)
27	InsElemRejectsElementOutsideUniverse	Элемент вне универсума вызывает std::out_of_range
28	DelElemRemovesElement	Удаление элемента
29	DelElemRejectsElementOutsideUniverse	Элемент вне универсума вызывает std::out_of_range
30	IsMemberRejectsElementOutsideUniverse	Элемент вне универсума вызывает std::out_of_range
31	CopyConstructorCreatesIndependentSet	Копия независима от оригинала
32	AssignmentCopiesSet	Присваивание копирует множество
33	SelfAssignmentIsSafe	Самоприсваивание безопасно
34	EqualityRequiresSameUniverseAndElements	Равенство требует одинаковых универсумов и элементов
35	PlusElementAddsCopy	Объединение с элементом возвращает копию
36	PlusElementRejectsOutsideUniverse	Элемент вне универсума вызывает std::out_of_range
37	MinusElementRemovesCopy	Разность с элементом возвращает копию
38	MinusElementRejectsOutsideUniverse	Элемент вне универсума вызывает std::out_of_range
39	UnionCombinesElements	Объединение множеств
40	UnionRejectsDifferentUniverses	Разные универсумы вызывают std::invalid_argument
41	IntersectionKeepsCommonElements	Пересечение множеств
42	IntersectionRejectsDifferentUniverses	Разные универсумы вызывают std::invalid_argument
43	ComplementInvertsMembership	Дополнение инвертирует принадлежность
44	ComplementDoesNotChangeOriginal	Дополнение не меняет оригинал
45	InputStreamReadsCharacteristicVector	Ввод элементов через пробел
46	OutputStreamPrintsElements	Вывод элементов через пробел
47	EmptySetPrintsNothing	Пустое множество ничего не выводит

Заключение

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

- Реализован класс TBitField — битовое поле с эффективным хранением битов в массиве элементов unsigned int. Реализованы все методы доступа, битовые операции и операторы ввода/вывода.
- Реализован класс TSet — множество на основе характеристического вектора. Реализованы все теоретико-множественные операции, методы доступа и операторы ввода/вывода.
- Написано 47 тестов с использованием Google Test, покрывающих основной функционал и краевые случаи. Все тесты успешно проходят.
- Освоен фреймворк Google Test, изучены принципы написания модульных тестов.

Литература

Список источников, используемых при выполнении работы (если есть).

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/
3. Кормен Т.Х., Лейзерсон Ч.И., Ривест Р.Л., Штайн К. Алгоритмы: построение и анализ
4. Ахо А.В., Хопкрофт Д., Ульман Д.Д. Структуры данных и алгоритмы.
5. Дасгупта С., Пападимитриу Х., Вазирани У. Алгоритмы.
6. Левитин А.В. Алгоритмы: введение в разработку и анализ.
7. Седжвик Р. Фундаментальные алгоритмы на C++.
8. Буч Г. Объектно-ориентированный анализ и проектирование с примерами приложений. – М.: Вильямс, 2010.

Страуструп Б. Программирование: принципы и практика с использованием C++, 2-е изд. – М.: Вильямс, 2016. – 1328

Приложение